Towards the Versioning of LLM-Agent-Based Software

Chengwei Liu, Lyuye Zhang*, Xiufeng Xu, Wenbo Guo, Yang Liu
Nanyang Technological University
Singapore
{chengwei.liu,lyuye.zhang}@ntu.edu.sg

Abstract

Large Language Models (LLMs) are redefining software practices by introducing agents that adapt rapidly to new data, tasks, and contexts. Conventional version control, optimized for source code updates, struggles to capture the evolving states of LLMs, prompts, training data, and agent behaviors. This paper proposes a multi-artifact approach that marries traditional code repositories with specialized registries for model weights, datasets, and prompt templates, tied together by metadata-rich descriptors. Through analogies to familiar computing layers and real-world scenarios, we illuminate the challenges of non-deterministic outputs, layered dependencies, and shifting user expectations. Our framework supports both "strict modes" for reproducibility in high-stakes contexts and "fluid modes" allowing iterative improvements with lower risk. By unifying versioning policies and tools across technical, organizational, and user-centric dimensions, we aim to foster more robust, transparent, and ethically informed LLM-agent systems that can reliably evolve alongside their expanding capabilities.

ACM Reference Format:
Chengwei Liu, Lyuye Zhang*, Xiufeng Xu, Wenbo Guo, Yang Liu. 2025. Towards the Versioning of LLM-Agent-Based Software. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3696630.3728714

1 Introduction

Large Language Models (LLMs) have become increasingly prevalent in AI-driven applications, transforming user interactions through natural language dialogues and complex decision-making processes [5]. LLM-based agents, in particular, integrate powerful language models with auxiliary components, such as knowledge bases, plugins, or environment interfaces, to continuously adapt their behaviors in response to new contexts and data [12, 15]. This dynamic, learning-oriented paradigm distinguishes such agents from conventional software modules that rely on fixed code paths and predetermined logic.

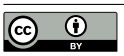
Despite their benefits, the continuous evolution of LLM agents poses significant challenges for teams aiming to maintain reliability and traceability. Traditional version control solutions (i.e., Semantic Versioning [10]), which are designed primarily for static code changes, struggle to capture the multitude of factors that drive agent behavior, including model weights, training data, and orchestration policies [7, 11]. Inadequate versioning of these artifacts can lead to inconsistent deployments, hinder rollbacks in the face of errors, and obscure the root causes of unexpected outputs. Consequently, a comprehensive, fine-grained versioning strategy is essential to ensure reproducibility, manage complexity, and build trust in LLM-agent-based systems [6].

Conventional version control solutions, though highly effective for static code bases, often fail to capture the complexities inherent in LLM-agent-based software. In these systems, multiple interconnected components, ranging from model weights and training data to orchestration logic and external interfaces, evolve continuously and interdependently [8, 13, 14]. As a result, developers struggle to reliably roll back to known stable states, trace the exact cause of unexpected behaviors, or ensure compatibility across evolving subsystems. This lack of holistic versioning hinders reproducibility, increases operational overhead, and diminishes the trust and agility needed to keep pace with the rapid innovation in LLM agents.

Although it is clear that traditional versioning methods fall short for LLM-agent-based software, devising a new solution is no trivial feat. One core challenge is *granularity*: while conventional systems track file-level changes, LLM agents need to account for shifting model parameters, evolving prompt templates, and real-time knowledge updates, each of which potentially requires their own versioning schemes. A second hurdle involves *interdependencies* among components that evolve at different paces or in different ways (e.g., a rapidly fine-tuned model versus a more static orchestration layer). Third, the *continuous and often non-deterministic* nature of LLM training and multi-agent interactions complicates rollback procedures and stability guarantees, as each change in training data or agent behavior can introduce new emergent effects [6]. Furthermore, ensuring *transparency and accountability* in such systems, particularly in distributed or multi-team environments, demands specialized tooling and protocols to certify that all relevant artifacts remain consistent, traceable, and auditable over time [9].

To navigate these obstacles, this paper outlines a multi-faceted versioning framework designed to accommodate the distinctive requirements of LLM-agent-based software. We begin by mapping out the core architectural components where versioning is most critical, highlighting how each evolves and interacts. We then propose a unified, fine-grained versioning strategy that tracks changes across all relevant artifacts, such as model checkpoints, training data, agent orchestration logic, and prompt templates, while ensuring backward compatibility and reproducibility. Finally, we discuss the broader implications of adopting such a scheme, including potential benefits, challenges, and open questions for future research.

*Lyuye Zhang is the corresponding author.



FSE Companion '25, Trondheim, Norway

ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3728714

2 Analogy Between Traditional Software Systems and LLM-Agent-Based Systems

Before detailing a versioning solution, it helps to draw parallels between the layers of a traditional computing stack and their counterparts in an LLM-agent-based environment. Over decades, hardware, firmware, operating systems, programming languages, and software applications have each developed robust versioning mechanisms for maintaining stability and compatibility. By mapping these established layers onto the components of an LLM-agent system, we clarify how existing practices might be extended or reimagined to handle the fluid, data-driven nature of AI models and agent orchestration.

• **Hardware ↔ Base LLM Architecture.** In traditional systems each hardware revision defines a stable execution environment. This environment includes an instruction set, a performance profile and low-level interfaces that higher-level software can rely on without modification. A frozen LLM checkpoint such as *GPT-4-0314* also plays such an identical role. Its weights, tokenizer and inference API become fixed components for prompts, adapters and orchestration workflows. Releasing a newer checkpoint, for example *GPT-4-0613*, requires dependent artefacts to update compatibility tags and undergo revalidation. Viewing base-model updates in this way shows why precise tracking of model versions and configurations is essential. Even small changes at this foundational level can cascade into unpredictable effects across the entire agent ecosystem [3, 4].

• **Firmware ↔ Training Data & Fine-Tuning.** Firmware customises generic hardware for a particular device context, and in the LLM stack, domain-specific data and fine-tuning layers play the same role. A compact set of adapter weights or LoRA deltas is "flashed" onto the frozen base model, injecting industry knowledge (e.g., medical jargon, legal style) without altering the underlying checkpoint. Each fine-tune run is versioned like a firmware image, explicitly linked to the base-model revision that it was trained against, and can be rolled forward or back to restore a known behavioural profile. This framing lets teams manage specialisation artefacts independently while preserving a clear compatibility contract between the core model and its customised variants.

• **Operating System ↔ Agent Platform.** Traditional operating systems define standardized resource management and core services that all applications rely on. LLM-agent platforms such as LangChain [1] and LangGraph [2] fulfill the same function by coordinating interactions among base models, prompt templates and external tool integrations. When a platform is updated, its new interfaces or orchestration logic can become incompatible with existing agent workflows or plugin implementations unless those components are explicitly versioned and validated against the updated platform release.

• **Programming Language ↔ Prompt Design.** In traditional software a programming language provides the syntax and semantics that developers use to express application logic. Prompt design in LLM-agent systems serves an equivalent role by specifying templates, few-shot examples and structured input to guide the model's responses [16]. Adjusting prompt phrasing or the selection of examples can produce substantial changes in agent behavior and therefore requires versioning safeguards similar to those applied when language syntax evolves.

• **Software/Apps ↔ Business-Specific Agents.** In a traditional stack the "application" is the top-level deliverable that bundles business logic, UI, and integrations. In an LLM stack, that role is filled by a business-specific agent package, which is a versioned bundle of prompt templates, tool calls, guard-rails, and orchestration code that surfaces the functionality to end-users (e.g., a support-desk chatbot or a RAG analyst). Like an app binary, the agents are unit-tested, semantically versioned, and released as the contract of record for the business service. Maintaining version control for these business-specific agents ensures consistent performance across evolving model or platform components [12].

• **External APIs/Tools ↔ Tools for Agents.** In conventional software, applications rely on external libraries and APIs to add new capabilities. In LLM-agent systems, plugins, search services and auxiliary AI modules serve the same purpose. When versions of these tools drift from what the agent expects, small mismatches can cause degraded performance or outright failures. Ensuring each external tool is versioned and validated alongside the core model and prompts prevents runtime errors and maintains overall agent reliability.

3 Divergences in LLM-Agent Version Management

Though Section 2 drew analogies between traditional software layers and LLM-agent components, the *nature of versioning* in LLM-based systems differs in several critical ways. By distilling the version-control problem into four core questions, ① Provenance, ② Change awareness, ③ Comparability, and ④ Auditability, we expose how and why traditional workflows fail, and what an LLM-aware mechanism must deliver.

Q1. (Provenance) What artefacts must a release identifier bind? Traditional software version tags secure a compiled binary and its library dependencies. In contrast, LLM agents derive their behavior from a complex ensemble of interdependent artefacts, including the frozen model checkpoint, fine-tune or LoRA adapter weights, prompt templates, retrieval or RAG indices, safety-rule documents, tool and API schemas, and the container image itself. Omitting any one of these can break reproducibility, since a change in, for example, a prompt may invalidate a safety policy. To address this, an LLM-agent version identifier must capture cryptographic hashes and signatures for every constituent artefact in a single unified manifest.

Q2. (Change Awareness) When does a modification warrant a new version? Teams working on code-centric projects usually overlook trivial edits like comments or formatting and bundle releases into weekly or monthly cycles. LLM-agent systems, by contrast, can be highly sensitive to small adjustments. Changing a single word in a prompt or applying a tiny adapter update may affect accuracy, bias, or latency far more than a large code overhaul. These agents also receive very frequent updates, with fine-tuning running hourly and retrieval indexes rebuilding at similar intervals. Sometimes a new version is needed solely because of data changes, such as incoming user feedback or updated policy files, even when no code has been modified. Version bumps must therefore be based on measurable shifts in behavior, for instance when accuracy or bias metrics cross predefined thresholds, rather than on the number

of lines changed or a fixed release schedule. This ensures that each new version genuinely represents a user-visible improvement.

Q3. (Comparability) How can two versions be compared when outputs are probabilistic? In deterministic systems, exact-match or checksum validation ensures that identical inputs on the same build yield identical outputs. LLM agents cannot offer such consistency because their sampling mechanisms, including temperature and top-p strategies, introduce token-level variability, and model providers may update weights without changing the API version. Consequently, version tags must capture every aspect of the stochastic configuration, such as sampling parameters and random seeds. Regression tests should be based on acceptable variance thresholds across relevant performance metrics rather than on simple pass-fail criteria. By focusing on allowable variation, normal noise should be properly filtered out while highlighting genuine regressions.

Q4. (Auditability) What runtime context is indispensable for replay or audit? In conventional systems, versioning alone suffices for post-deployment auditing because the static binary and its logged inputs fully determine behavior. LLM agents, however, incorporate ephemeral execution details, such as dialogue context, retrieval results, and dynamic agent negotiations, that lie outside the scope of ordinary logs. Ignoring these transient elements leaves gaps in any reconstruction of past runs, which undermine both compliance and debugging efforts. Therefore, a versioning mechanism for LLM agents should extend beyond static artefact tagging to include enriched, time-stamped audit records that capture sufficient runtime context alongside the composite version tag. Such comprehensive audit trails should be preserved to close the provenance loop, which ensures that every decision can later be traced back to the exact system state in which it was made.

4 Versioning LLM-Agent-Based Systems

To reconcile the rapid, heterogeneous evolution of LLM agents with the consistency guarantees of mature version control, we introduce a unified tagging scheme paired with a GPG-signed manifest. This design simultaneously captures the full provenance of every artefact, automates semantic change detection, enforces compatibility constraints at merge time, and produces an immutable record of environment and performance metadata. By leveraging familiar DevOps and MLOps mechanisms, such as Git tags, model registries, CI hooks, and manifest signing, our approach could embed these four properties (provenance, change awareness, comparability, and auditability) into existing workflows with minimal friction.

4.1 Schema for LLM-Agent Versioning

Our versioning schema extends standard Semantic Versioning by appending a concise behavioural fingerprint paired with a cryptographically signed manifest. Each release is identified by a composite tag of `MAJOR.MINOR.PATCH+BASE-ADAPTER-PROMPT[#BUILD]`, where:

- `MAJOR.MINOR.PATCH` follow SemVer rules: breaking changes, new features, and bug fixes.
- `BASE` is the frozen LLM checkpoint ID (e.g., `gpt4-0314`).
- `ADAPTER` denotes the fine-tuning or LoRA module tag (or `none`).
- `PROMPT` is the first eight hex digits of the prompt template's SHA–256 hash.
- `#BUILD` (optional) references CI metadata (such as Git SHA, Docker digest, dataset tag, evaluation metrics, random seed, etc.).

A GPG-signed `bundle.toml` manifest accompanies each tag, recording the exact Git commit SHA, model registry URI, dataset snapshot, and prompt hashes. By binding every artefact to a stable digest, the manifest becomes the single source of truth: checking out a tag and verifying its signature guarantees identical retrieval and replay of code, weights, data, and prompts. This unified schema enables automated bump decisions, compatibility enforcement, and tamper-proof audit trails, while integrating seamlessly into existing Git and CI/CD workflows.

4.2 Ensuring Provenance, Change Awareness, Comparability, and Auditability

In our framework, each release tag resolves to a GPG-signed manifest that cryptographically binds the Git commit SHA, model registry URI, dataset snapshot, and prompt hash, guaranteeing provenance and precise reproduction of any past agent state for debugging or regulatory audits. Change awareness is automated via a pre-merge CI hook that diffs the new manifest against the previous release and applies Semantic Versioning rules: a base-model swap (for example, from `1.1.0+gpt4-0314_...` to `2.0.0+gpt4-0613_...`) triggers a MAJOR bump with summary "model 0314→0613", adapter or prompt updates induce a MINOR bump, and metadata-only tweaks result in a PATCH bump, so engineers know which regression suites to rerun. To enforce comparability, the manifest embeds a compatibility matrix of valid `BASE-ADAPTER-PROMPT` tuples, and CI rejects any merge proposing unsupported combinations or failing accuracy, bias, or latency thresholds. Finally, auditability is provided by the manifest's signature and optional build metadata (including Git SHA, Docker digest, dataset tag, evaluation metrics, and random seed), creating a tamper-proof record that supports internal reviews, compliance reporting, and automated multi-agent version negotiation.

Our composite tagging schema and signed manifest also naturally dovetail with the popular Model Context Protocol (MCP) and and agent-to-agent (A2A) orchestration. In an MCP environment where models, data, and inference workflows are managed as discrete, versioned components, every stage can consume the same `MAJOR.MINOR.PATCH+BASE-ADAPTER-PROMPT[#BUILD]` tag (and its manifest) to fetch the exact code, weights, data, and prompts it needs, run reproducible experiments, and promote only fully validated bundles from development through staging to production. In A2A scenarios where autonomous agents interact by invoking each other's capabilities, the tag functions as a concise contract. Each agent advertises its version tag and embedded compatibility matrix, allowing callers to verify support for required model checkpoints, adapters, and prompt schemas before executing a remote call. If an incompatibility is detected, agents can automatically negotiate by downgrading, launch a companion instance with the requested version, or escalating to a human operator. By aligning seamlessly MCP and A2A scenarios, our framework could ensure end-to-end consistency and interoperability across distributed LLM ecosystems.

5 Discussion

• **Limitations and threats to validity.** Our framework assumes that every component—code, model checkpoint, dataset, prompt—can be captured and hashed reliably. In real deployments, streaming data sources or continuously fine-tuned models may evade complete snapshotting. The bump rules for MAJOR, MINOR and PATCH versions rest on human judgment, so subtle changes in adapter behavior or prompt wording can lead to inconsistent tagging. We have validated this approach only in small, controlled pipelines rather than large, federated systems, so its reproducibility and compatibility guarantees may weaken in more complex or opaque environments.

• **Practical challenges to adoption.** Integrating model benchmarks and prompt regression tests into continuous integration can extend build times from minutes to hours, a burden for teams without dedicated hardware. Retaining multiple large model files and dataset versions increases storage costs and management overhead. Smaller organizations may lack the skills to create and verify signed manifests or to maintain a compatibility matrix across many adapter and prompt versions. Migrating legacy workflows that treat code, data and prompts as separate silos will require significant engineering effort and careful change management.

• **Future research directions.** Future work should explore how versioning can evolve from a static tagging system into a dynamic governance layer that anticipates and mitigates risk. One promising direction is the development of AI-driven versioning tools that analyze proposed changes across code, data, models, and prompts to predict compatibility issues before deployment, reducing human guesswork and preventing runtime failures. Equally important is the design of decentralized provenance frameworks—potentially leveraging blockchain or similar tamper-proof ledgers—to record every manifest and version negotiation event in a way that is verifiable across organizational boundaries. We should also investigate ethical versioning mechanisms that continuously monitor performance and fairness metrics and automatically revert or quarantine versions when bias or safety violations are detected. Finally, creating cross-platform standards and a unified API for version discovery, negotiation, and enforcement will be critical to enable seamless interoperability among heterogeneous agents and LLM vendors. Together, these advances will shift version control from a reactive label to an adaptive, policy-driven system that underpins trustworthy, large-scale AI ecosystems.

6 Conclusion

Classic software versioning lacks the flexibility to manage the dynamic, interdependent nature of LLM-agent-based ecosystems, where models, data, and prompts can all evolve independently and non-deterministically. To address these gaps, this paper proposes bundling code, models, and data snapshots under metadata-rich commits, striking a balance between rapid iteration and reliable traceability. Our analysis underscores the value of strict modes for regulated environments and fluid modes for less critical updates. In the future, AI-driven compatibility checks, decentralized provenance, and cross-platform standards could further stabilize LLM-agent systems and foster more transparent, ethically aligned applications.

Acknowledgment

This research is supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1 (RG96/23). It is also supported by the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG2-GC-2023-008); by the National Research Foundation Singapore and the Cyber Security Agency under the National Cybersecurity R&D Programme (NCRP25-P04-TAICeN) and CyberSG R&D Cyber Research Programme Office; and by the National Research Foundation, Prime Minister's Office, Singapore under the Campus for Research Excellence and Technological Enterprise (CREATE) programme. Any opinions, findings and conclusions, or recommendations expressed in these materials are those of the author(s) and do not reflect the views of National Research Foundation, Singapore, Cyber Security Agency of Singapore as well as CyberSG R&D Programme Office, Singapore.

References

[1] 2025. LangChain. https://www.langchain.com/ [Online; accessed 2025-04-25].
[2] 2025. LangGraph. https://www.langchain.com/langgraph [Online; accessed 2025-04-25].
[3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
[5] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of Artificial General Intelligence: Early experiments with GPT-4. arXiv:2303.12712 [cs.CL] https://arxiv.org/abs/2303.12712
[6] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. How is ChatGPT's behavior changing over time? *Harvard Data Science Review* 6, 2 (2024).
[7] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. 2022. Machine Learning Operations (MLOps): Overview, Definition, and Architecture. arXiv:2205.02302 [cs.LG] https://arxiv.org/abs/2205.02302
[8] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems* 36 (2023), 51991–52008.
[9] M Mesarčík, Sára Solárová, J Podroužek, and Maria Bielikova. 2021. Stance on The Proposal for a Regulation Laying Down Harmonised Rules on Artificial Intelligence–Artificial Intelligence Act. *Kempelen Institute of Intelligent Technologies* (2021).
[10] Tom Preston-Werner. 2025. Semantic Versioning 2.0.0 | Semantic Versioning. https://semver.org/ [Online; accessed 2025-04-25].
[11] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. *Advances in neural information processing systems* 28 (2015).
[12] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. arXiv:2303.17580 [cs.CL] https://arxiv.org/abs/2303.17580
[13] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
[14] Yi Tay, Mostafa Dehghani, VQ Tran, X Garcia, J Wei, X Wang, HW Chung, S Shakeri, D Bahri, T Schuster, et al. 2022. UL2 20B: An Open Source Unified Language Learner. *Computation and Language* 19, 2 (2022), 131.
[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] https://arxiv.org/abs/2210.03629
[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.